AI ENGINEERING ASSESSMENT

Invoice Canonicalization Agent

Deterministic first. AI only for uncertainty.
Human approvals become cheaper future knowledge.

TIER 1
Approved lookup
no LLM

TIER 2
Retrieval / AI
bounded

TIER 3
Human learning
governed

Production-oriented design + tested reference implementation

CHALLENGE ACCEPTANCE

The brief shows three different runs. All three must converge.

3 / 3	18 / 18	0
runs converge	observed variants	LLM calls

We turned every supplied description into an executable acceptance contract: 3 runs × 6 products = 18 observed variants.

#	RUN 1 observed	RUN 2 observed	RUN 3 observed	REQUIRED RESULT
1	Steel Accessories	Highlife Components	Steel Fittings	**Highlife Components**
2	Unstoppable Sneakers	Performance Sneakers	Athletic Shoes	**Performance Sneakers**
3	Polarbear T-Shirt	White Tee	Logo T-Shirt	**White Tee**
4	Grizzly T-Shirt	Beige Tee	Logo Tee	**Beige Tee**
5	El Camino Shorts	Casual Shorts	Quick-Dry Shorts	**Casual Shorts**
6	Black Socks	Crew Socks	Athletic Crew Socks	**Crew Socks**

Acceptance rule: every supplied variant resolves to the same tenant-scoped canonical product ID - by approved exact lookup, not generation.

Source: supplied assessment, page 1

CHALLENGE PROOF

The actual invoice input also resolves 6/6 deterministically

The raw descriptions on the supplied invoice map directly to the requested canonical taxonomy with approved aliases and zero model calls.

INVOICE

LOGO

Testinger GmbH
123 Test Street
Testingen, Testern, 0815
+00 1234 5678 00
testinger.ts
email@testinger.ts

01. Juli 2025

19283746552

Due in 15 days

BILL TO
Mr Obertestinger
Recipient Corp.
123 Empfängergasse
0815 Recipstan
+00 1234 8876 0912,
recip@recipcopr.co

SHIP TO
Mr Obertestinger
Recipient Corp.
123 Empfängergasse
0815 Austria
+00 1234 8876 0912,
recip@recipcopr.co

DESCRIPTION	QTY	UNIT PRICE	TOTAL
Highlife Steel Accessories	1	10	10.00
Sneaker "Unstoppable"	2	11	22.00
T-Shirt White "Polarbear"	3	12	36.00
T-Shirt Beige "Grizzly"	4	13	52.00
Shorts "El Camino"	5	14	70.00
Socks, black	6	15	90.00

Remarks / Payment Instructions:

SUBTOTAL	280.00
DISCOUNT	13.00
SUBTOTAL LESS DISCOUNT	267.00
TAX RATE	20.00%
TOTAL TAX	53.40
SHIPPING/HANDLING	12.00
Balance Due	**$ 332.40**

Company Signature

Client Signature

Deterministic replay

Highlife Steel Accessories	→	**Highlife Components**	NO LLM
Sneaker "Unstoppable"	→	**Performance Sneakers**	NO LLM
T-Shirt White "Polarbear"	→	**White Tee**	NO LLM
T-Shirt Beige "Grizzly"	→	**Beige Tee**	NO LLM
Shorts "El Camino"	→	**Casual Shorts**	NO LLM
Socks, black	→	**Crew Socks**	NO LLM

6 / 6
raw invoice mappings
deterministic replay

0
LLM calls
challenge path

PASS
extraction gate
6/6 row equations

280=280
subtotal check
Decimal

Together with the 18/18 three-run acceptance matrix, this proves the supplied task end-to-end. Unseen-model quality is evaluated separately.

DESIGN TARGET

What the task actually requires

A reproducibility problem first: the same booking text should resolve to the same canonical result across repeated runs and client contexts.

ISSUES IN THE BRIEF

- same invoice text should map to one reproducible description
- direct LLM generation can vary between repeated runs
- different clients and business partners can use different taxonomies
- the solution must specify how consistency is enforced

DESIGN RESPONSE

- approved canonical IDs + tenant-scoped aliases
- deterministic exact-match path before any AI call
- bounded AI only for unseen or uncertain descriptions
- human approval promotes trusted future exact aliases
- guardrails + regression tests make behavior auditable

DESIGN PRINCIPLE **Determinism is a data and governance property - not a prompt-engineering trick.**

TRUSTED RAG

The database is the textbook - not the model

Both deterministic lookup and the AI fallback use the same human-approved, tenant-scoped knowledge store.

Tier 1: exact lookup

Known wording matches an approved alias and returns a stable canonical product ID.

No LLM call.

HUMAN-APPROVED

Knowledge Store

canonical products · approved aliases
tenant + partner scope · taxonomy version

Tier 2: approved RAG evidence

Unknown wording retrieves only from approved catalog evidence before bounded AI proposes or abstains.

Single source of truth

The catalog is controlled by the organization, not model memory.

Open-book AI

The model can use the approved textbook; it cannot invent trusted facts.

Receipt / traceability

Decisions keep alias, taxonomy, tenant, prompt/model and review evidence.

Key rule: unapproved model output never becomes retrieval evidence until a human promotes it.

PROCESS · 1

Step 1 - validate invoice evidence before canonicalization

Known or wild, the same extraction and arithmetic quality gate runs first. No product enters canonicalization from dirty invoice evidence.

Every line — tamed or wild — passes the same extraction and arithmetic gate.

quantity × unit price = line total · Σ line totals = subtotal · discount / tax / shipping reconciled separately

Only clean product-line evidence moves forward into deterministic lookup or the uncertainty path.

PROCESS · 2

Step 2 - known products take the fast lane; unknowns are routed deliberately

Approved cases are deterministic; unseen descriptions are investigated, not silently guessed.

Known / Tamed

Exact approved alias → stable canonical product ID. No LLM call.

Unknown / Wild

Fast lane blocked. Unknown is routed deliberately — not treated as a failed invoice.

Approved knowledge first

Before AI, the system checks approved canonical knowledge and past human decisions.

PROCESS · 3

Step 3 - retrieval, bounded AI, and human approval resolve the unknown

Tier 2 gathers evidence and proposes a candidate. Tier 3 approves knowledge before anything becomes trusted.

Retrieve evidence → bounded AI proposal → human approves / edits / rejects

An unapproved AI guess never becomes trusted knowledge. Human approval is the promotion step.

PROCESS · 4

Step 4 - approval turns uncertainty into reusable deterministic knowledge

Once approved, the same description becomes an exact alias match on future invoices.

0 LLM calls on future exact matches

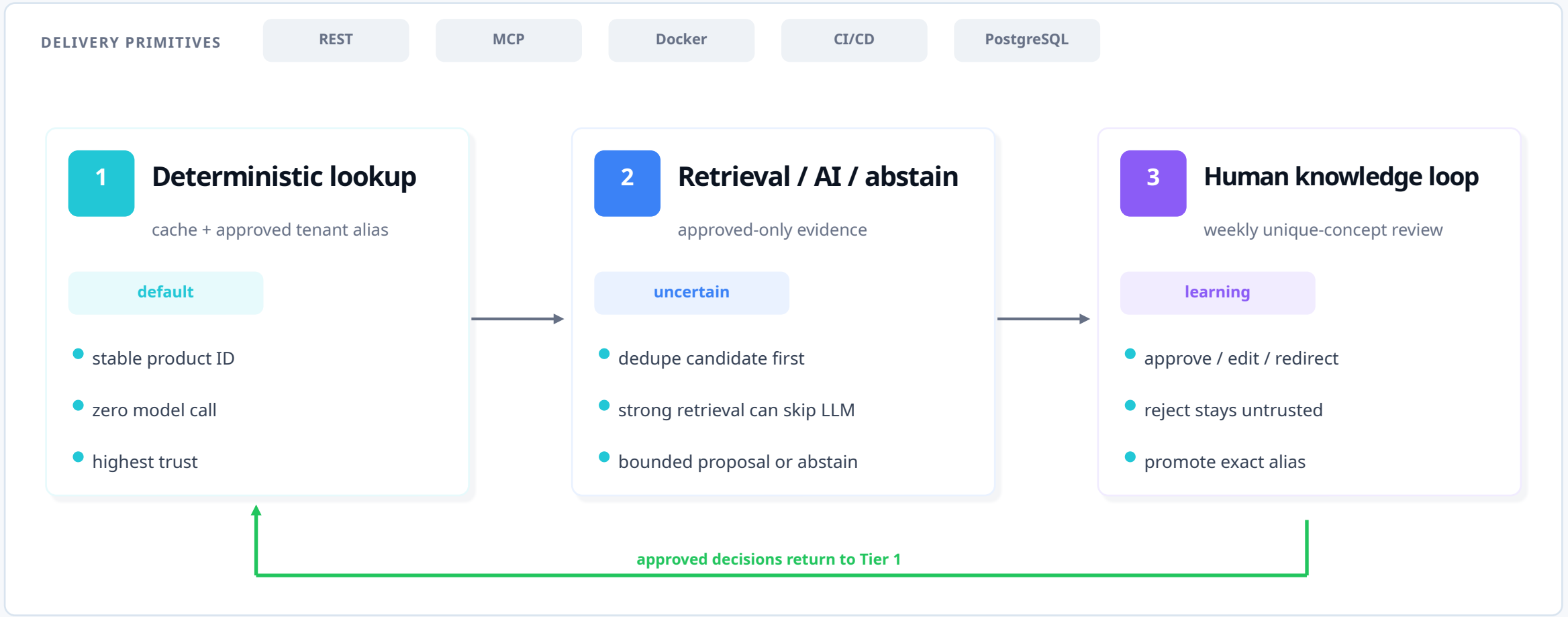
0 human reviews on future exact matches

↓ marginal model cost as the alias registry grows

ARCHITECTURE

Three-tier canonicalization architecture

That story maps directly to three production tiers; delivery primitives stay outside the business logic.



Story -> code: Approved Lookup = approved aliases | Uncertainty Lab = approved retrieval + bounded AI / abstention | Human Approval = review queue -> knowledge promotion.

VALUE LOOP

One AI proposal can remove future model calls

Model spend follows unique unresolved concepts, not repeated invoice volume; approval turns uncertainty into exact knowledge.

01
New unknown
Black Leather Jacket Midnight

02
Bounded AI
1 proposal: Black Leather Jacket

03
Repeated 26x
0 additional model calls

04
Human approval
promote trusted alias

05
Next invoice
exact alias; 0 model calls

26 observations of same unknown

1 model proposal while pending

0 future calls after approval

Business takeaway: every approved human decision bends the cost curve down by converting uncertainty into deterministic Tier 1 knowledge.

INVOICE CANONICALIZATION AGENT

RESULTS

Measured reliability: verified gates, not vanity metrics

The final code now passes static checks and keeps golden-set evidence separate from production-accuracy claims.

111	18/18	11/11	PASS	0
automated tests	**challenge variants**	**routing cases**	**Ruff + mypy**	**unsafe auto-accepts**
offline suite	3 runs converge	curated regression	static checks	target 0

What the results prove

- all 3 supplied runs converge to the same six canonical descriptions
- 26 repeated unknown observations collapse to 1 proposal
- 0 unexpected LLM calls on the golden regression set
- 0 model-review bypasses and unsafe inputs can abstain
- 83% branch-aware coverage and package installs in isolation

What we do NOT claim

- 18/18 challenge acceptance = ML accuracy (it is deterministic alias coverage)
- 11 curated cases predict production accuracy at scale
- vector RAG outperforms lexical retrieval (not benchmarked)
- offline fixtures validate real provider latency / cost / drift

Current verified gate: 111 tests · 83% branch-aware coverage · 18/18 supplied variants · Ruff PASS · mypy PASS · 11/11 curated routing cases · 0 unsafe auto-accepts.

TAKEAWAY

Reproducible by default.
AI only where uncertainty remains.

The system solves the challenge deterministically, then uses bounded AI only for the unknown tail.

DETERMINISTIC

Approved aliases return the same canonical product every time.

GOVERNED

Unapproved AI proposals never become trusted knowledge silently.

LEARNING

Human approval converts uncertainty into future exact matches.

“The goal is not to make the LLM more consistent. It is to need the LLM less often.”

18/18 supplied variants · 6/6 raw invoice · 11/11 golden routing · Ruff PASS · mypy PASS · 111 tests

APPENDIX

Technical depth when the interviewer asks

Unknown lifecycle, review workflow, tenant isolation, delivery packaging, testing, retrieval, security, CI/CD, and limitations.

- 18-stage quality contract
- test matrix
- security + cost
- retrieval / RAG
- review data model
- limitations

HUMAN LOOP

Humans approve knowledge - not invoices

A weekly queue contains unique unresolved concepts with evidence, impact, and flexible actions.

PENDING KNOWLEDGE REVIEW

27 unique concepts

71,482 lines affected

Priority	Observed variants	Proposal	Score	LLM	Occurrences	Status
HIGH	Air Runner Pro Black AirRunner Pro blk	**Performance Sneakers**	98.2%	Yes	**18,400**	approve
HIGH	Black Leather Jacket Midnight	**Black Leather Jacket**	73.0%	Yes	**26**	waiting
MED	Athletic crew socks	**Crew Socks**	97.8%	No	**127**	waiting

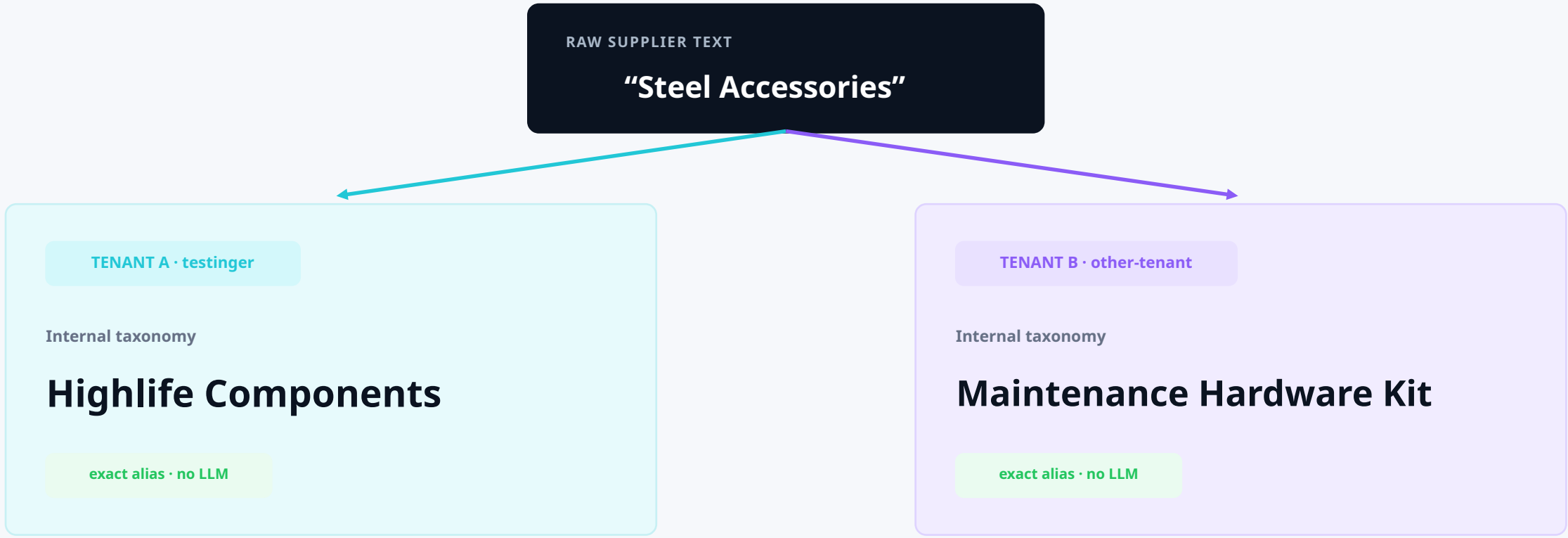
Reviewer actions: Approve existing | Redirect | Create new | Edit + approve | Reject | Defer

Completed decisions leave the active queue but remain archived + audited.

MULTI-CLIENT

Tenant taxonomies prevent cross-client collisions

The raw string is not a global key: alias lookup is scoped by tenant + partner + language.



Uniqueness scope: tenant_id + partner_id + normalized_alias + language

Authentication binds the request to a tenant; PostgreSQL RLS is a second defense-in-depth layer.

SCOPE

Core solution vs. production packaging

This avoids over-engineering the story: the business logic stays small; the rest is delivery discipline.

CORE BUSINESS LOGIC

- canonical product IDs
- tenant-scoped approved aliases
- normalization + retrieval policy
- bounded AI for unknowns
- human knowledge promotion

OPERATIONAL GUARDRAILS

- extraction arithmetic gate
- auth + tenant binding
- budgets / timeouts / retries
- validation + abstention
- audit + monitoring

DELIVERY PRIMITIVES

- REST API
- MCP integration boundary
- Docker / Compose
- GitHub CI/CD
- PostgreSQL migrations

Interview rule: lead with the first column. Use the other two only when asked how you would productionize it.

Internal quality sequence - 18 machine-checked stages

Appendix detail: this is an implementation contract, not the primary business architecture.

01-06

Input + evidence

- auth + tenant binding
- file validation
- deterministic parsing
- Decimal line gate
- model extraction fallback
- persist invoice envelope

07-10

Deterministic state

- normalize text
- version-safe cache
- approved exact alias
- atomic pending claim

11-14

Uncertainty handling

- approved-only retrieval
- policy scoring
- bounded generation
- output/security validation

15-18

Governance

- staged review queue
- human action
- knowledge promotion
- audit + metrics

Testing strategy

Offline determinism for CI; live-provider evaluation remains opt-in.

Layer	What it proves	Property
Unit / contracts	normalization, money, budgets, prompt schemas	fast / deterministic
Integration	repository, canonicalization, review promotion	real application wiring
Document metamorphic	PDF / XLSX / DOCX / JSON / CSV / TXT	same business result across formats
AI fixtures	stored prompt examples + model outputs	no API-spend CI, stable regressions
Adversarial	prompt injection, PII/financial leakage, cross-tenant, malformed output	safe failure / abstention
Concurrency	same unknown arriving simultaneously	one candidate / one proposal
Packaging	wheel build + isolated install smoke	delivery integrity

Challenge acceptance: 3 supplied runs × 6 descriptions = 18/18 variants → the same 6 canonical outputs, with 0 LLM calls.

Security, reliability, and cost guardrails

These protect the product logic; they do not decide the canonical product.

SECURITY

- tenant-bound auth
- PostgreSQL RLS
- party PII kept out of prompts
- prompt-injection guard
- CSV formula neutralization

RELIABILITY

- Decimal financial fields
- line arithmetic gate
- commercial reconciliation
- schema + attribute validation
- immutable audit history

COST / LATENCY

- exact alias before retrieval
- dedupe before model
- max calls + dollar budget
- timeouts + bounded retries
- cache by taxonomy version

Retrieval / RAG boundary

Use the simplest retrieval method that meets false-match, latency, and recall targets.

CURRENT ASSESSMENT

- approved aliases only
- sequence similarity
- token overlap
- character trigrams
- category + protected attributes
- margin vs. second candidate

SCALE-UP OPTION

- SemanticScoreProvider interface exists
- PostgreSQL schema is pgvector-ready
- benchmark Recall@K + false match first
- measure latency + memory before enabling
- do not index unapproved model suggestions

Anti-feedback-loop rule: staging data never becomes retrieval evidence until a human promotes it.

Review data model and state transitions

Local CSV is an editable projection; production persistence is relational.

Candidate record

- normalized candidate key
- source variants + occurrence count
- affected value by currency
- retrieval / policy scores
- model + prompt provenance
- risk flags + evidence
- human overrides + notes

Human-editable final status

- waiting
- approve existing
- redirect
- create new
- edit + approve
- reject
- defer

Processed rows leave the active work queue but remain in archive + audit history.

Concurrency: one active candidate per tenant + partner + normalized description.

Limitations and next validation

What still needs evidence before a real production rollout.

Area	Current evidence	Next validation
Production accuracy	11 curated cases are regression evidence, not population statistics	Build a blind labelled dataset across clients/categories/languages
Threshold calibration	current thresholds are configuration, not statistically calibrated probabilities	Optimize to a business target such as false auto-resolution < 0.1%
Live provider	offline fixtures prove contracts but not real latency/cost/drift	Benchmark candidate models on a blind labelled set; pin the best quality / cost / latency trade-off
Semantic retrieval	pgvector seam exists but is not benchmarked as better	Compare Recall@K / latency / false-match against lexical hybrid
Release environment	Ruff + mypy pass; Docker remains a separate release-environment check	Keep Docker build / smoke test as an independent release CI job
Operational UX	CSV proves workflow but is not a polished reviewer product	Replace with dashboard when reviewer volume justifies it

Delivery and demo commands

Everything needed to inspect, test, demonstrate, and package the solution.

`./run_assessment.sh`	Offline quality gate: tests, coverage, evaluation, package smoke
`make interview-demo`	Challenge 3-run acceptance + three-tier lifecycle + machine-readable evidence
`make review-demo`	Generate the editable pending-knowledge CSV
`make review-process`	Apply reviewer decisions, archive completed items
`make assess-full`	Strict release gate including Ruff, mypy, Docker

Assessor-facing artifacts

- README.md
- docs/assessor_summary.md
- docs/invoice_data_model.md
- reports/final_delivery_check.md
- docs/verified_results.md
- docs/production_validation_plan.md
- presentation/PPTX + PDF

Version 1.3.0